Lecture 3

Data Analysis at the Unix Command line

BMMB 852: Applied Bioinformatics

The birth of an epiphany

Last week I visited Italy (Arezzo) saw a statue and had an epiphany.



I now better understand biological data formats.

(Epiphany is an “Aha!” moment. Often, an epiphany begins with a small, everyday occurrence or experience.)

Guido de Arezzo, 991 AD, Arezzo, Italy



Invented musical staff notation that replaced the old style "neumatic notation".

The plaque said: **The time needed to train a church singer dropped from 10 years to 1 year.**

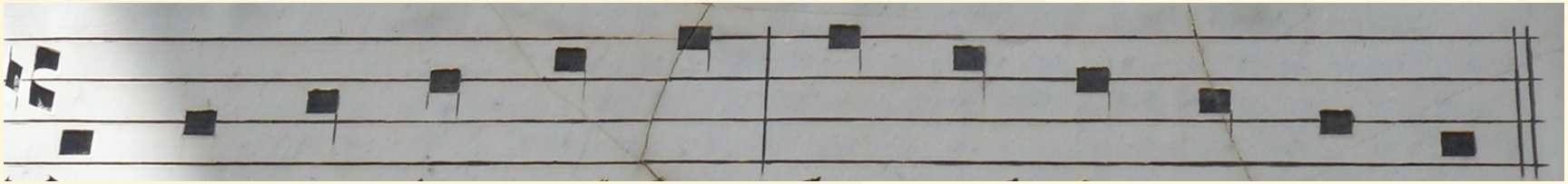
Old style neumatic notations

	**Scandicus**	Three notes ascending
	**Climacus**	Three notes descending
	**Torculus**	down-up-down
	**Porrectus**	up-down-up

etc. everything is a "special case" with a weird name, note how "scientific" it feels, almost a systematic enumeration

New style staff notations

Each note represents a single note



What are the fundamental differences between the two notations?

The second notation captures the "real" data.

The first type of notation is a "brute force", and ultimately unworkable attempt to find and enumerate all variations, give each a scientific name, tabulate into a big book etc.

But it is modeling the "wrong" data.

The second notation finds the elemental building blocks and uses those

The Epiphany: Part 1

Data representation is the primary limiting factor to discoveries

An inefficient formalism

- Makes you work harder
- Imposes hidden limits on what you can do
- Continuously hinders progress
- You are never certain: *is it me* or *is it the system* that keeps me from reaching my goals
- Its greatest sin: **the more you know - the more limiting it gets**

Another example (stick with Italy)

Are roman numerals easy to use?

```
I   II   III   IV   V
```

...discussion...

- what would make people want to adopt this notation?

Are arab numerals easy to use?

```
I   II   III   IV   V   VI   VII   VIII  IX    X
```

versus

```
0    1    2    3    4    5    6    7     8     9
```

...discussion...

Roman numberals are incomprehensible for large numbers

```
MMMMMMMMMMMMMMMMMMMMCDLVI
```

vs

```
19456
```

The innovation is the decimal place!

Now we can build formulas!

Science was stuck until a better representation came along.

Roman numberals are easy if you do super simple things.

Otherwise are atrocious

Roman numberals limit what we can achieve

Epiphany: Part 2

Bioinformatics data is stuck in the era of roman numerals.

Current data formats are "simple" in the most "simplistic" sense.

The reason we will need to work harder than needed is that we have to overcome the limitations built into the representation system.

Bioinformatic is the "art" of doing calculus with roman numerals.

How to fix it?

1. How can be objectively assess that we have a problem?
2. How can we tell that the current formalism for numbers is optimal?
3. How to tell apart another Guido the Arezzo from a kook?

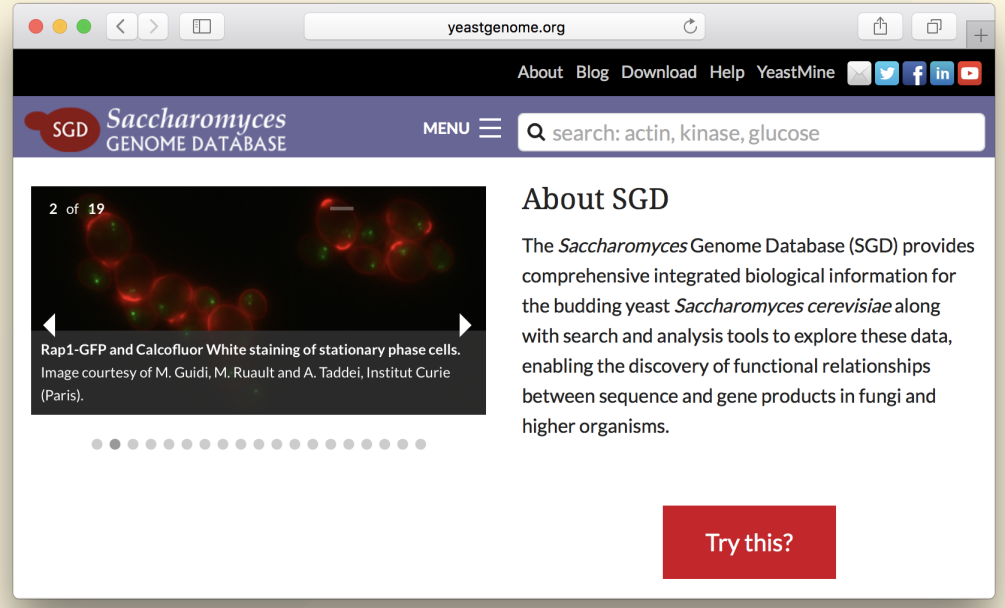
...discuss...

The remainder of this presentation can be also read in the chapter "Data analysis with Unix"

Unix data analysis skills are universally useful!

You may be able to make use of them in other contexts as well.

Web based databases



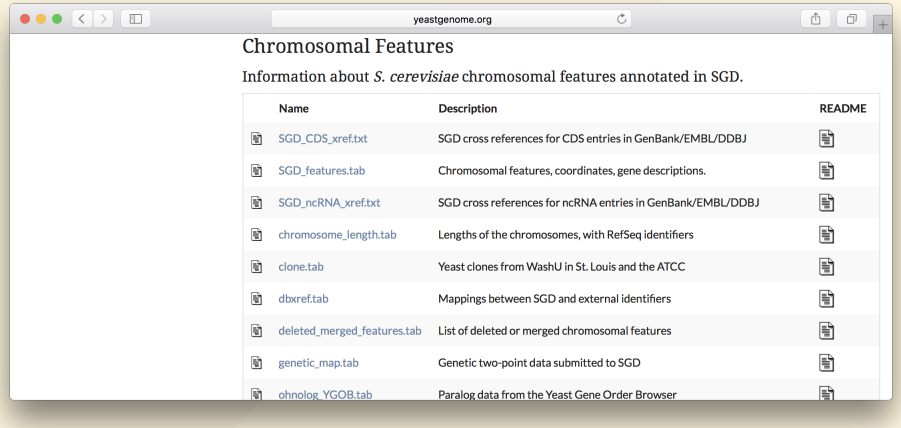
Websites can be handy for exploration. Finding specific data may be suprisingly convoluted

Even simple questions can be impossible answer via a web interface

- How many genomic features are there?
- How many on each chromosome and strand?
- How many features are reliably determined?
- What is the lenght of the coding regions relative to the entire genome?

Strategy: keep a record

Once you find information - save it into a text file -> this will be your *script* eventually.



When you found the URL address, save it into a file:

```
http://downloads.yeastgenome.org/curation/
chromosomal_feature/SGD_features.tab
```

Get the "specification file"

Typically there are two files, a description file and the data file

- **SGD_features.README**
- **SGD_features.tab**

Getting online data

For every lecture create a new folder to work in.

```
mkdir lec03
cd lec03
```

The `wget` command can download a remote file.

```
wget http://downloads.yeastgenome.org/curation/chromosomal
```

Where are you and what have you got here:

```
pwd
ls
```

Paging through files

You can step through a file with the `more` command:

```
more SGD_features.tab
```

keypress commads within `more`:

- `SPACE` or `f` go forward, next page
- `b` go backward, previous page
- `/word` search for a word
- `/` repeats the search for the last word
- `h` help

Think in terms of streams

Instead of treating the file as a UNIT think of it as a stream of information. Open a stream with `cat`

```
cat SGD_features.tab
```

A stream can be made to flow from the left to right from one tool into the next via the pipe `|` character.

Limit the stream by piping into another tool. See what the `head` and `tail` tools do:

```
cat SGD_features.tab | head -1
```

Command line usage tips

Your command line is smarter than Siri! (though that is a very low bar to pass).

1. Recall previous commands with the `ARROW` keys
2. Start writing something then ask your command line to "auto fill" the rest by pressing `TAB`
3. If nothing happens then there are conflicting ways to auto-fill. Double tap `TAB` to see all options

It takes a little practice but you can become very efficient at entering commands.

Simple commands

How many lines, words and characters in this stream:

```
cat SGD_features.tab | wc
```

prints the lines, words and characters:

```
16454  425719 3264490
```

Use flags to customize `wc`. Show the number of lines:

```
cat SGD_features.tab | wc -l
```

A large variety of tools

You don't have to remember them all. The ones you use will stick in your mind. `grep` finds matches in files:

```
cat SGD_features.tab  | grep YAL060W
```

will print (see handbook for unwrapped lines):

```
S000000056      ORF     Verified        YAL060W BDH1    (R
S000036089      CDS                                     YA
```

You can make it color the match with:

```
cat SGD_features.tab | grep YAL060W --color=always
```

Flags can change tools quite a bit

What does not match the word "Dubious" (a weird word that in this file indicates uncertainty about the feature).

Use `grep -v`:

```
cat SGD_features.tab | grep -v Dubious | head -5
```

How many "dubious" and "non-dubious" features:

```
cat SGD_features.tab | grep Dubious | wc -l
cat SGD_features.tab | grep -v Dubious | wc -l
```

How do I store results in a new file?

The `>` character is the "redirection". Instead of the screen it goes into a file

```
cat SGD_features.tab | grep YAL060W > match.tab
```

now check how many files do you have:

```
match.tab
SGD_features.tab
```

You now a subset of the origina data stored in the new file `match.tab`

How do select columns?

It looks like this file uses the feature type (column 2) ORF for protein coding genes. You will need to cut the second field (by tabs).

```
cat SGD_features.tab | cut -f 2 | head
```

prints:

```
ORF
CDS
ORF
CDS
ARS
telomere
telomeric_repeat
```

How do I build my commands

Build your commands one step at a time, always checking that you are on the right track.

Write one command, run through a "limiter" (`head`) then add a new command, rerun it and so on.

Ensure that you understand what the command is doing "so far"

```
cat SGD_features.tab | head
cat SGD_features.tab | cut -f 2 | head
cat SGD_features.tab | cut -f 2 | grep ORF | head
```

Wise man say: "every problem can be solved with some kind of sorting"

Sorting places identical consecutive entries next to one another.

Keep the feature types:

```
cat SGD_features.tab | cut -f 2 > types.txt
```

Sort the feature types:

```
cat types.txt | sort | head
```

sort + uniq

The solution to suprisingly number of questions

The `uniq` command collapses consecutive identical words into a single entry.

```
cat types.txt | sort | uniq | head
```

How many types of features in this file:

```
cat types.txt | sort | uniq | wc -l
```

Prints:

```
44
```

Sort + Uniq

The challenge often is to recognize that the problem can be modeled as a `sort` + `uniq` action.

`uniq -c` also reports the number of times the item has been seen:

```
cat types.txt | sort | uniq -c | head
```

prints the counts and types:

```
 352 ARS
 196 ARS_consensus_sequence
7074 CDS
  50 LTR_retrotransposon
```

Learn the sort + uniq pattern

It can answer many questions

We will use it a lot